R.M.K
GROUP OF ENGINEERING INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

R.M.K
GROUP OF
INSTITUTIONS

Please read this disclaimer before proceeding:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.

22IT957
DEVOPS

Department: IT

Batch/Year: 2023-27 / III

Created by:

Dr. A. Anna Lakshmi , ASP/IT/RMKEC

Ms. N. Abirami ,AP/IT/RMDEC

Date: 27.11.2025

R.M.K GROUP OF INSTITUTIONS

Table of Contents

S.NO.	CONTENTS	SLIDE NO.
1	CONTENTS	5
2	COURSE OBJECTIVES	6
3	PRE REQUISITES (COURSE NAMES WITH CODE)	7
4	SYLLABUS (WITH SUBJECT CODE, NAME, LTPC DETAILS)	8
5	COURSE OUTCOMES (6)	11
6	CO- PO/PSO MAPPING	12
7	LECTURE PLAN –UNIT III	14
8	ACTIVITY BASED LEARNING –UNIT III	16
9	CROSSWORD PUZZLE	16
10	QUIZ LINK	17
11	TEST YOURSELF	18
12	LECTURE NOTES – UNIT III	19
13	ASSIGNMENT - UNIT III	51
14	PART A Q & A	52
15	PART B Q s (WITH K LEVEL)	55
16	SUPPORTIVE ONLINE CERTIFICATION COURSES	61
17	REAL TIME APPLICATIONS IN DAY-TO-DAY LIFE AND TO INDUSTRY	62
18	ASSESSMENT SCHEDULE	63
20	PRESCRIBED TEXT BOOKS & REFERENCE BOOKS	64
21	MINI PROJECT SUGGESTIONS	65

R.M.K GROUP OF INSTITUTIONS

2. COURSE OBJECTIVES

OBJECTIVES:

- Understand the concepts of DevOps and the issues it
- resolves Learn the DevOps tools set
- Learn to Develop automation using Maven
- Understand Continuous Delivery and Continuous
- Deployment Understand Docker Containerization

R.M.K GROUP OF INSTITUTIONS

3. PRE REQUISITES

PRE-REQUISITE CHART

22IT957 DEVOPS

- DATA STRUCTURES
- Software Engineering
- Cloud Computing

4. 22IT957 **DEVOPS (Lab Integrated)**

L	T	P	C
2	0	2	3

OBJECTIVES:

The Course will enable the learners:

- Understand the concepts of DevOps and the issues it resolves
- Learn the DevOps tools set
- Learn to Develop automation using Maven
- Understand Continuous Delivery and Continuous Deployment
- Understand Docker Containerization

UNIT I INTRODUCTION TO DEVOPS 6+6

What Is DevOps? Culture, principles & benefits, Architecture, SDLC Life Cycle & DevOps LifeCycle, Workflow and Principles, Tools, CI, CD and CD Pipelines – Version Control Systems (e.g., Git, GitHub/GitLab/Bitbucket) Linux Introduction Basic Commands, Scripting – Toolsets – SSH, Curl, CAT, Sudo, Man, apt-get *

,

List of Exercises:

1. Prepare an application preferably in Spring Boot with following features like

a. Maven Build
b. Unit Testing
c. Application Functionality
d. DockerFile
e. App Configuration
f. And finally, the application checked in to GitHub Repo

2. Install Jenkins with following plugins

a. Maven
b. Java
c. GitHub
d. Artifact Repo
e. Pipeline
f. SonarQube

3. Create a simple Spring Boot application that calculates the sum of two numbers

R.M.K GROUP OF INSTITUTIONS

4. 22IT957 **DEVOPS (Lab Integrated)** **L T P C**
2 0 2 3

UNIT II TOOLS SET 6+6

Maven Build Management, Goals, Profiles, Plugins, LifeCycles, Configuration, Parent/Child - SCM Tools - GitHub, Init, CheckIn, Merge, Pull, Push, Local and Remote Repo, Pull Request, Tagging Strategy – Unit Testing – Unit Testing scripts - Artifact Repository - Release Management aligned Repos, Private and Public Repos Monitoring - Tools like nagios to assist in monitoring and managing the deployed instances.

List of Exercises:

1. Create a Spring Boot App Using Maven
2. Configure and Setup a CI in Jenkins with the application developed earlier. And enable following configurations.
   a. Trigger the CI – GitHub Hooks
   b. Generate the CI Report and publish the same as Email ,
   c. Test Case Report
   d. Dynamic Build ID Generation
   e. Post and Pre build activities.
3. Create Automated Unit and Functional (QA) Testing Scripts using TestNG & Selenium

UNIT III TESTING AUTOMATION 6+6

Maven with Unit / Integration / Performance Testing - Report Generation and Configuration-Jenkins Configuration – CD – Pipelines – Schedule and Hooks Run.

List of Exercises:

1. Configure and Setup a CD in Jenkins with the application developed earier.
   a. Enable JenkinsFile for automating the build process
   b. Enable Unit testing automation in the CD Pipeline.
2. Enable Functional testing automation in the CD Pipeline.

UNIT IV DEPLOYMENT AND MONITORING – DOCKER 6+6

Docker Introduction, Images, Containers, Docker Hub, Links, Volume, Network, Interactive Sessions , Docker REST API - Persistence Volume and Claim, Replica Factor, Services, Pods, Deployments, logs, Kubernetes, K8 – Helm Framework, Single and Cluster Mode, Secrets.

4. 22IT957 **DEVOPS (Lab Integrated)** **L T P C**
2 0 2 3

List of Exercises:

1. Install and Configure Docker
2. Configure and Setup a CD in Jenkins with the application developed earier.

- Add DockerFile Build Process for the application in scope.
- Build and Deploy the same
- Add the Docker Clean up, Image Creation, and creating instance of Containers via CD Process. (using pre-build scripts)

UNIT V DEPLOYMENT AND MONITORING - JENKINS 6+6

Jenkins Configuration, Jobs - Continuous Integration, Continuous Delivery and Continuous Deployment Configuration
Jenkins CD->Integration with SonarQube, Artifact Repository, Docker Hub & Kubernetes via REST API , Logging and Monitoring - Prometheus, Grafana, Nagios ,

List of Exercises:

1. Install and Nagios
2. Apply monitoring principles to measure

a. Build Time
b. Unit & Integration Testing Time
c. Resources Consumed by Jenkins
d. Deployment Time
e. Analyze the logs files for any anomalies
f. Verifying ACLs and Security based access rights.
g. Network Performance
h. Overall Serer Storage, Performance etc

TOTAL: 30+30=60 PERIODS

5.COURSE OUTCOME

Course Code	Course Outcome Statement	Cognitive/ Affective Level of the Course Outcome	Expected Level of Attainment
Course Outcome Statements in Cognitive Domain			
C702.1	Understand the concept of DevOps Architecture.	Apply K3	60%
C702.2	Apply the DevOps Tools in real time applications	Analyse K4	60%
C702.3	Build Maven with Unit, Integration and Performance Testing	Understand K2	60%
C702.4	Deploy and monitor using Docker	Apply K3	60%
C702.5	Deploy and monitor using Jenkins	Analyse K4	60%
C702.6	Integrate DevOps workflows by leveraging continuous integration and continuous deployment (CI/CD) pipelines to enhance efficiency, collaboration, and quality in software development and operations.	Apply K3	60%
Course Outcome Statements in Affective domain			
C702.7	Attend the classes regularly	Respond (A2)	95%
C702.8	Submit the Assignments regularly.	Respond (A2)	95%
C702.9	Participation in Seminar/Quiz/ Group Discussion/ Collaborative learning and content beyond syllabus	Valuing (A3)	95%

R.M.K GROUP OF INSTITUTIONS

6.CO-PO/PSO MAPPING

Correlation Matrix of the Course Outcomes to Programme Outcomes and Programme Specific Outcomes Including Course Enrichment Activities

Course Outcomes (Cos)		Programme Outcomes (POs), Programme Specific Outcomes (PSOs)														
		PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
		K3	K4	K5	K5	K3/K5	A2	A3	A3	A3	A3	A3	A2	K3	K3	K3
C305.1	K3	3	2	1	1	3						3		3	3	3
C305.2	K4	3	3	2	2	3						3		3	3	3
C305.3	K2	2	1									2		3	3	3
C305.4	K3	3	2	1	1	3						3		3	3	3
C305.5	K4	3	3	2	2	3						3		3	3	3
C305.6	K3	3	2	1	1	3						3		2	2	2
C305.7	A2												3			
C305.8	A2								2	2	2		3			
C305.9	A3						3	3		3	3		3			
C305		3	3	2	2	3	1	1	1	3	3	3	3	3	3	3

UNIT 3

TOOL AUTOMATION

LECTURE PLAN – UNIT III

	UNIT I SOFTWARE PROCESS AND AGILE DEVELOPMENT						
Sl. No	TOPIC	NO OF PERIODS	PROPOSED LECTURE PEROID	ACTUAL LECTURE PERIOD	PERTAINING CO(s)	TAXONOMY LEVEL	MODE OF DELIVERY
1	Introduction to Maven	1			CO3	K2	MD2
2	unit tests using the Surefire Plugin	1			CO3	K3	MD2
3	Integration testing and configure the Failsafe Plugin.	1			CO3	K3	MD2
4	Performance testing tools like JMeter	1			CO3	K3	MD2
5	generate test and project reports using Maven plugins.	1			CO3	K3	MD2
6	Reports for continuous improvement.	1			CO3	K4	MD2
7	Configuration-Jenkins Configuration	1			CO3	K2	MD2
8	CD – Pipelines	1			CO3	K4	MD2
9	Schedule and Hooks Run.	1			CO3	K3	MD2

LECTURE PLAN – UNIT 3

ASSESSMENT COMPONENTS

- AC 1. Unit Test
- AC 2. Assignment
- AC 3. Course Seminar
- AC 4. Course Quiz
- AC 5. Case Study
- AC 6. Record Work
- AC 7. Lab / Mini Project
- AC 8. Lab Model Exam
- AC 9. Project Review

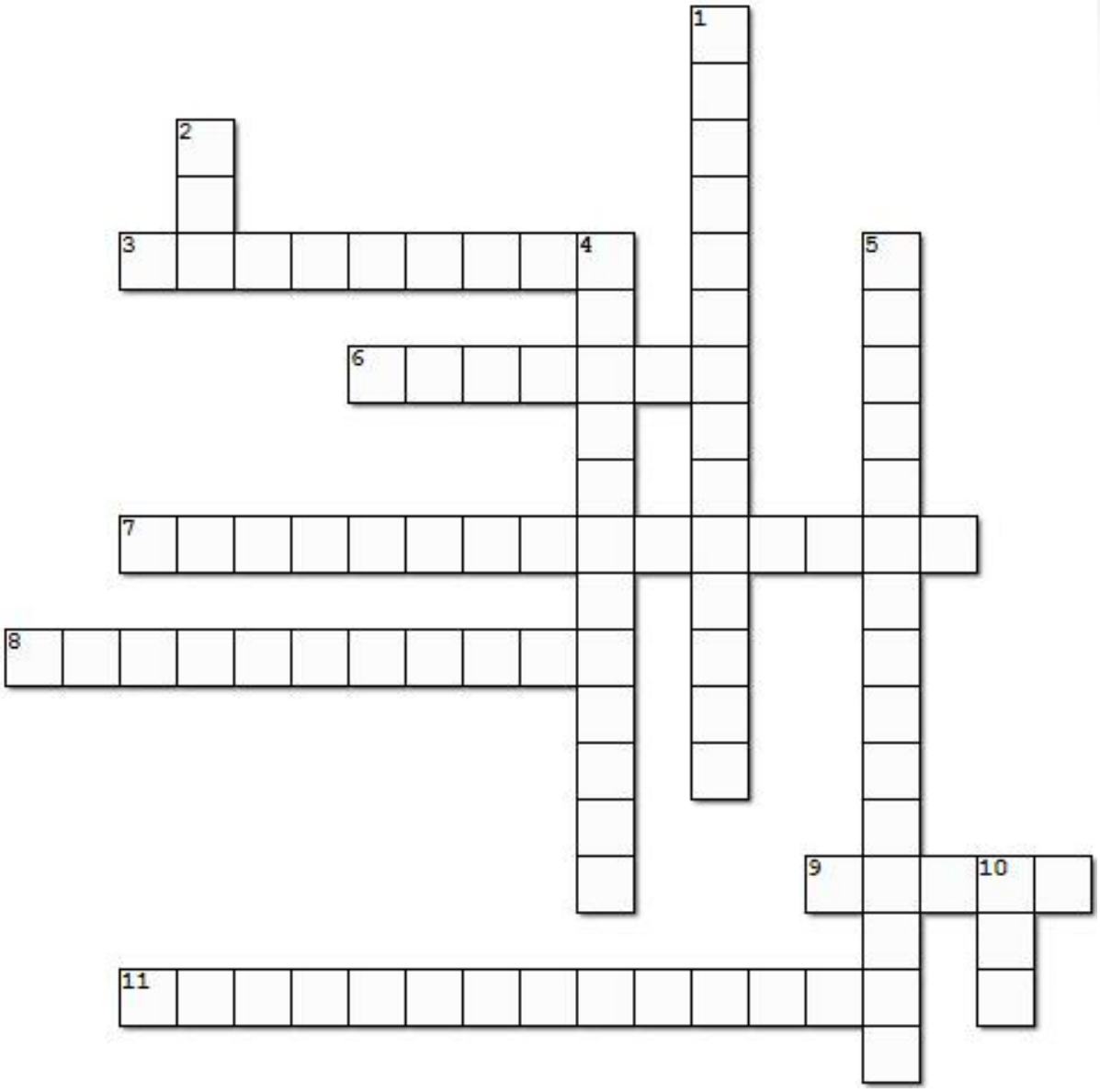
MODE OF DELEIVERY

MD 1. Oral presentation
MD 2. Tutorial
MD 3. Seminar
MD 4 Hands On
MD 5. Videos
MD 6. Field Visit

R.M.K GROUP OF INSTITUTIONS

ACTIVITY BASED LEARNING – UNIT I

COMPLETE THE PUZZLES GIVEN BELOW



Down

1. lightweight, efficient, low-risk, flexible, predictable approach
2. In XP, User requirements are expressed as
3. Inception,elaboration,construction,deployment
5. critical safety systems & defect free systems
6. Project risk factor is considered in which model

Across

4. XP design
7. The spiral model was originally proposed by
8. Requirement well understood
9. Stand-up meetings
10. Modelling tool
11. Process of correcting errors

ACTIVITY BASED LEARNING – UNIT I

QUIZ- LINK

Unit 3:

https://quizizz.com/admin/quiz/6417c38f432a57002135873c/devops?fromSearch=true&source=null

DEVOPS VIDEO link

https://youtu.be/slajN4ox7Eo?si=sCcz6AGsjvOBiimp

Test Yourself

1. **When was DevOps introduced?**

- A) 1999 and 2000
- B) 2003 and 2004
- C) 2007 and 2008
- D) 2010 and 2011

2 . **Who is the father of DevOps?**

- A) Patrick Debois
- B) Gene Kim
- C) David Farley
- D) Bridget Kravchenko

3 **What is the full form of Devops?**

- A) Drive and Operations
- B) Digital and Operations
- C) Development And Operations
- D) None of the above

4. **Identify the tool for DevOps?**

- A) Monit
- B) Nagios
- C) Jenkins
- D) All of the above

UNIT 3
TESTING AUTOMATION

OVERVIEW:

Testing is the process of checking the functionality of an application to ensure it runs as per requirements. Unit testing comes into picture at the developers' level; it is the testing of single entity (class or method). Unit testing plays a critical role in helping a software company deliver quality products to its customers.
Unit testing can be done in two ways: manual testing and automated testing.

Manual Testing	Automated Testing
Executing a test cases manually without any tool support is knownas manual testing.	Taking tool support and executing the test cases by using an automation toolis known as automation testing.
Time-consuming and tedious:Since test cases are executed byhuman resources, it is very slow and tedious.	Fast: Automation runs test cases significantly faster than human resources.
Huge investment in human resources: As test cases need to be executed manually, more testersare required in manual testing.	Less investment in human resources: Test cases are executed using automation tools, so less numberof testers are required in automation testing.
Less reliable: Manual testing is less reliable, as it has to account for human errors.	More reliable: Automation tests areprecise and reliable.
Non-programmable: No programming can be done to writesophisticated tests to fetch hiddeninformation.	Programmable: Testers can program sophisticated tests to bring out hidden information.

Maven Unit Testing

- Maven is a project management and comprehension tool that provides developers a complete build lifecycle framework. Development team can automate the project's build infrastructure in almost no time as Maven uses a standard directory layout and a default build lifecycle.
- In case of multiple development teams environment, Maven can set-up the way to work as per standards in a very short time. As most of the project setups are simple and reusable, Maven makes life of developer easy while creating reports, checks, build and testing automation setups.
- Maven provides developers ways to manage the following –
- Builds
- Documentation
- Reporting
- Dependencies
- SCMs
- Releases
- Distribution
- Mailing list
- To summarize, Maven simplifies and standardizes the project build process. It handles compilation, distribution, documentation, team collaboration and other tasks seamlessly. Maven increases reusability and takes care of most of the build related tasks.

R.M.K GROUP OF INSTITUTIONS

Maven Unit Testing

- **Maven Evolution**
- Maven was originally designed to simplify building processes in Jakarta Turbine project. There were several projects and each project contained slightly different ANT build files. JARs were checked into CVS.
- Apache group then developed **Maven** which can build multiple projects together, publish projects information, deploy projects, share JARs across several projects and help in collaboration of teams.
- **Objective**
- The primary goal of Maven is to provide developer with the following –
- A comprehensive model for projects, which is reusable, maintainable, and easier to comprehend.
- Plugins or tools that interact with this declarative model.
- Maven project structure and contents are declared in an xml file, pom.xml, referred as Project Object Model (POM), which is the fundamental unit of the entire Maven system. In later chapters, we will explain POM in detail.

Convention over Configuration

Maven uses Convention over Configuration, which means developers are not required to create build process themselves.

Developers do not have to mention each and every configuration detail. Maven provides sensible default behavior for projects. When a Maven project is created, Maven creates default project structure. Developer is only required to place files accordingly and he/she need not to define any configuration in pom.xml.

As an example, following table shows the default values for project source code files, resource files and other configurations. Assuming, ${basedir} denotes the project location –

- In order to build the project, Maven provides developers with options to mention life-cycle goals and project dependencies (that rely on Maven plugin capabilities and on its default conventions). Much of the project management and build related tasks are maintained by Maven plugins.

Features of Maven

- Simple project setup that follows best practices.
- Consistent usage across all projects.
- Dependency management including automatic updating.
- A large and growing repository of libraries.
- Extensible, with the ability to easily write plugins in Java or scripting languages.
- Instant access to new features with little or no extra configuration.
- **Model-based builds** – Maven is able to build any number of projects into predefined output types such as jar, war, metadata.
- **Coherent site of project information** – Using the same metadata as per the build process, maven is able to generate a website and a PDF including complete documentation.
- **Release management and distribution publication** – Without additional configuration, maven will integrate with your source control system such as CVS and manages the release of a project.
- **Backward Compatibility** – You can easily port the multiple modules of a project into Maven 3 from older versions of Maven. It can support the older versions also.
- **Automatic parent versioning** – No need to specify the parent in the sub module for maintenance.
- **Parallel builds** – It analyzes the project dependency graph and enables you to build schedule modules in parallel. Using this, you can achieve the performance improvements of 20-50%.
- **Better Error and Integrity Reporting** – Maven improved error reporting, and it provides you with a link to the Maven wiki page where you will get full description of the error.

JUnit

What is JUnit?

- JUnit is a unit testing framework for Java programming language. It plays a crucial role test-driven development, and is a family of unit testing frameworks collectively known as xUnit.
- JUnit promotes the idea of "first testing then coding", which emphasizes on setting up the test data for a piece of code that can be tested first and then implemented. This approach is like "test a little, code a little, test a little, code a little." It increases the productivity of the programmer and the stability of program code, which in turn reduces the stress on the programmer and the time spent on debugging.

Features of JUnit

- JUnit is an open source framework, which is used for writing and running tests.
- Provides annotations to identify test methods.
- Provides assertions for testing expected results.
- Provides test runners for running tests.
- JUnit tests allow you to write codes faster, which increases quality.
- JUnit is elegantly simple. It is less complex and takes less time.
- JUnit tests can be run automatically and they check their own results and provide immediate feedback. There's no need to manually comb through a report of test results.
- JUnit tests can be organized into test suites containing test cases and even other test suites.
- JUnit shows test progress in a bar that is green if the test is running smoothly, and it turns red when a test fails.

What is a Unit Test Case?

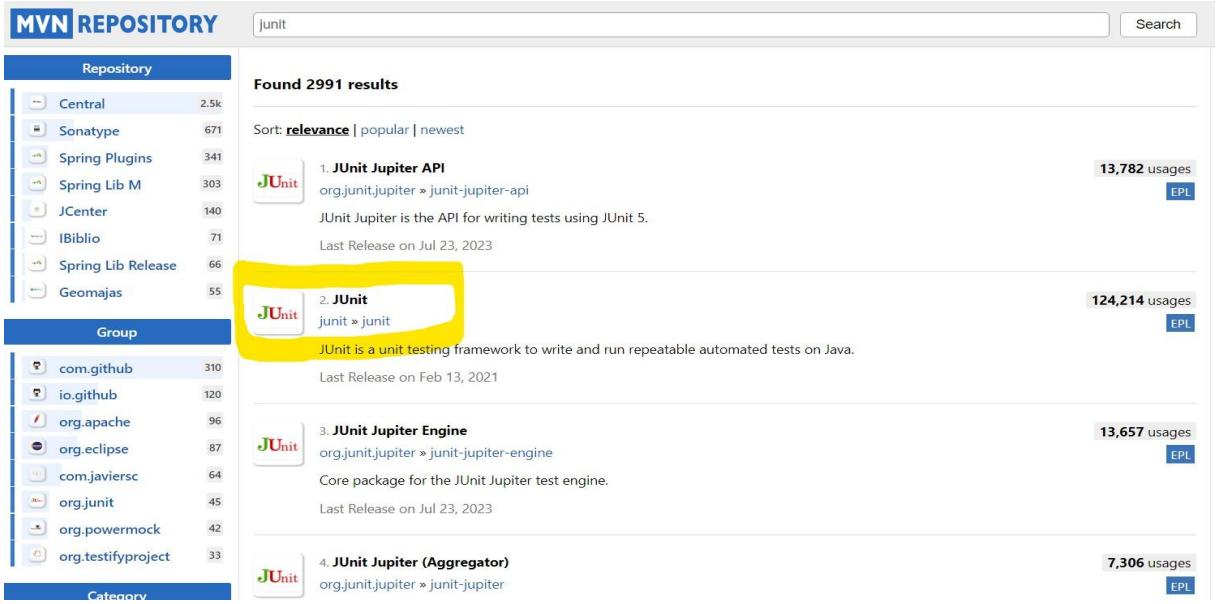
- A Unit Test Case is a part of code, which ensures that another part of code (method) works as expected. To achieve the desired results quickly, a test framework is required. JUnit is a perfect unit test framework for Java programming language.
- A formal written unit test case is characterized by a known input and an expected output, which is worked out before the test is executed. The known input should test a precondition and the expected output should test a post-condition.
- There must be at least two unit test cases for each requirement: one positive test and one negative test. If a requirement has sub-requirements, each sub-requirement must have at least two test cases as positive and negative.
- JUnit is a **Regression Testing Framework** used by developers to implement unit testing in Java, and accelerate programming speed and increase the quality of code. JUnit Framework can be easily integrated with either of the following:
- Eclipse
- Ant
- Maven

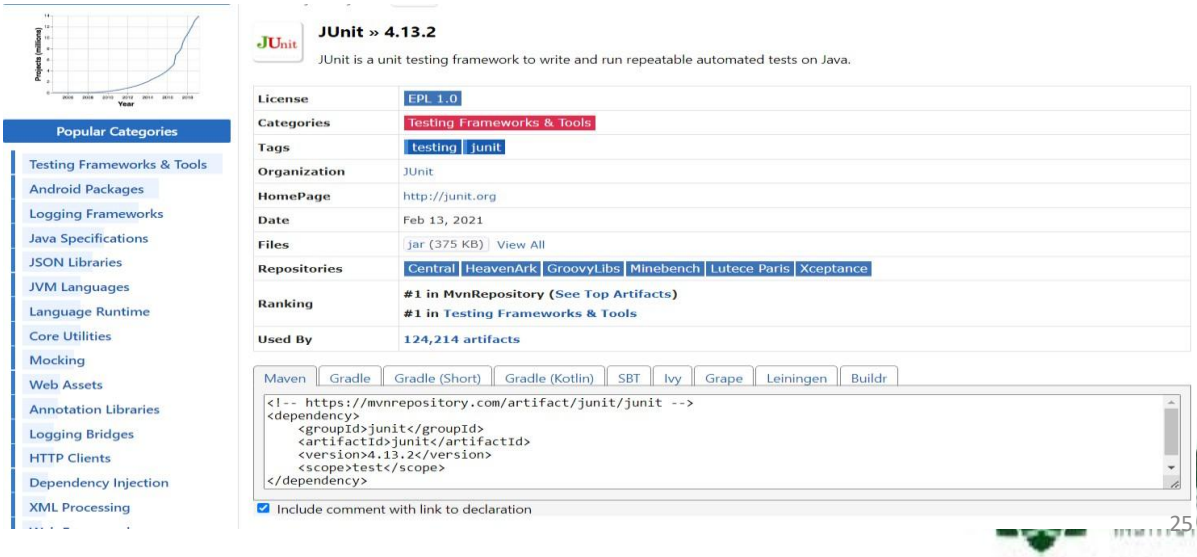
Test Suites

- A test suite bundles a few unit test cases and runs them together. In JUnit, both @RunWith and @Suite annotation are used to run the suite test.

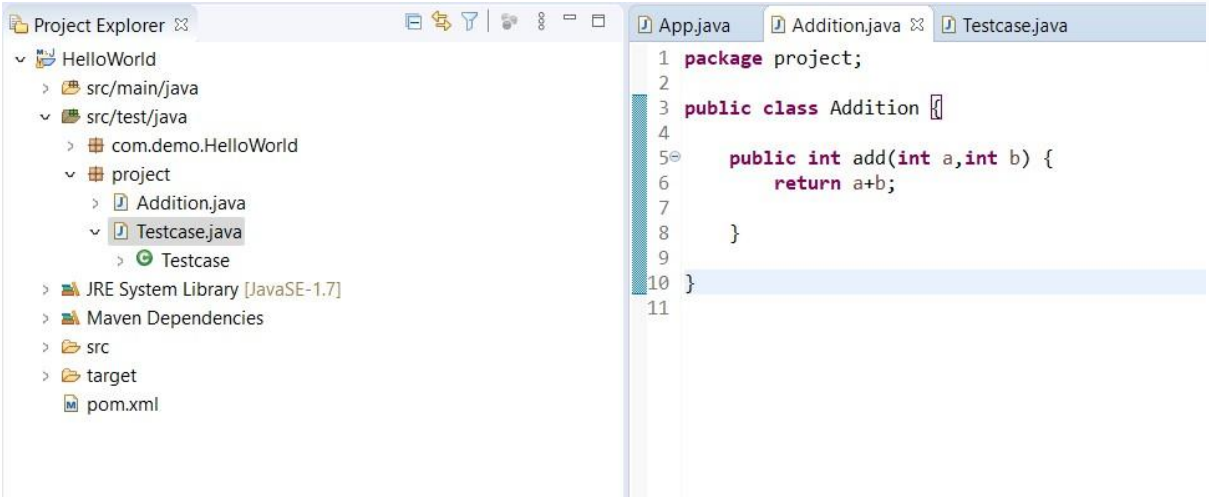
Steps to create maven project and to test with Junit (Unit Testing)

- 1. Download Eclipse and create a workspace to develop the code.
- https://www.eclipse.org/downloads/packages/installer
- 2. Create a maven project by choosing **maven-archtype-quickstart** archetype.
- A sample app which will display hello world in the console is created.
- 3. Add the dependencies if needed(JUnit) using the maven central repository.
- https://mvnrepository.com/repos/central

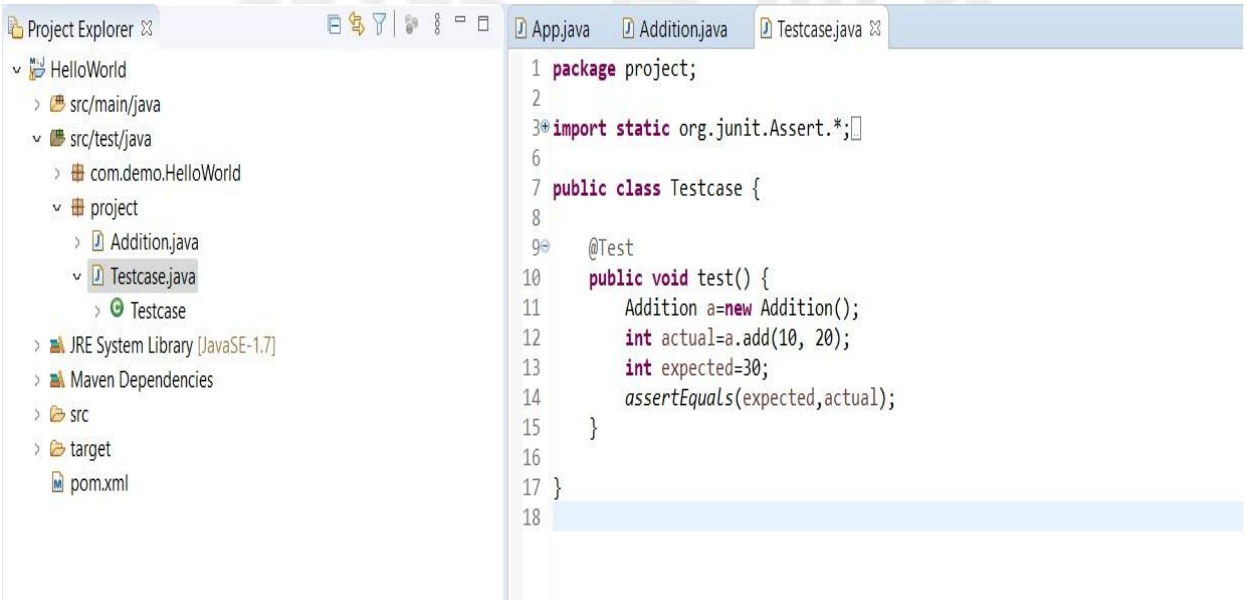




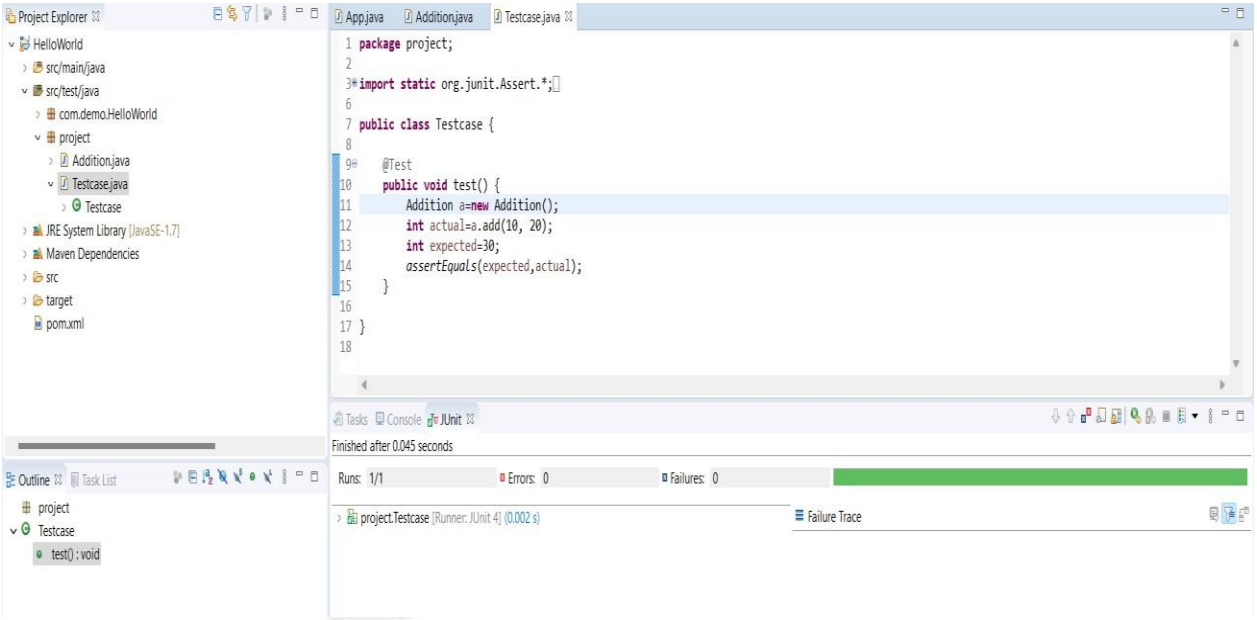
4. Now, In the folder **src/test/java** create a class which will hold the functionality of your application. For instance , consider you want to build the functionality of addition,

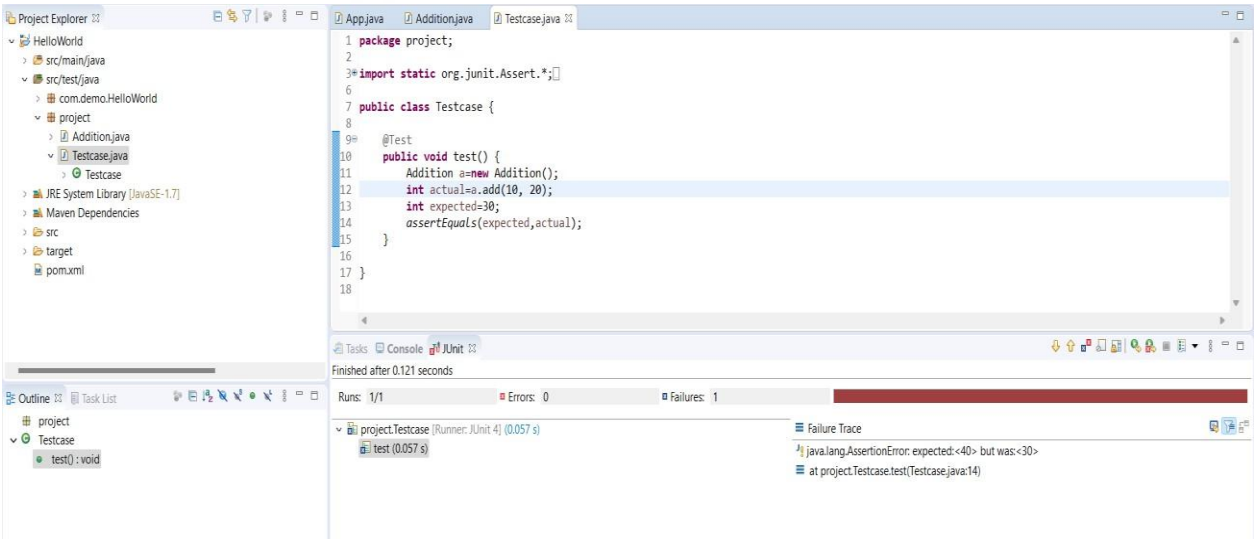


5. In the same folder, create JUnit class to provide the test case.



- Once implementing both the classes, the next step is to check for the functioning of the code.
- 6. Right click on the file and select **Run as -> Junit** .
- 7. If the expected and produced output are the same then green colour is indicated in the window, otherwise red colour will be indicated.





- Now, right click on the pom.xml file and select **Run as -> maven test** .It will provide you the details of build code. If it has this kind of error,

```
[INFO] ------------------------------------------------------------
[INFO] ------------------------------------------------------------------------
[INFO] BUILD FAILURE
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.574 s
[INFO] Finished at: 2021-06-13T19:26:45+05:30
[INFO] ------------------------------------------------------------------------
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.1:testComp
[ERROR] Source option 5 is no longer supported. Use 7 or later.
[ERROR] Target option 5 is no longer supported. Use 7 or later.
[ERROR] -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the followir
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoFailureException
```

- Then add this property in the pom.xml file.

```
15  <properties>
16    <maven.compiler.source>8</maven.compiler.source>
17    <maven.compiler.target>8</maven.compiler.target>
18  </properties>
```

- Now, follow the above step to run the code again

Maven with Unit Testing- Example

- Learn to *run JUnit tests using Maven Surefire Plugin*. We will learn to **run a single test, run only selected tests or run all the tests** in the project.
- Quick Reference
- *//Run all tests*
- $ mvn test
-
- *//Run a single test class*
- $ mvn -Dtest=TestClassOne test
-
- *//Run multiple test classes*
- $ mvn -Dtest=TestClassOne,TestClassTwo test
-
- *//Run a single test method*
- $ mvn -Dtest=TestClassOne#methodname test
-
- *//Run tests matching name 'testMethod' in all test classes*
- $ mvn -Dtest="*#testMethod" test
-
- *//Run tests matching name 'test*' in a test class*
- $ mvn -Dtest="TestClassOne#test*" test
-
- *//Rerun failing tests 2 times*
- mvn '-Dsurefire.rerunFailingTestsCount=2' -Dtest=ModuleTwoTests test
- **1. Setup**
- Begin with importing the latest version of maven-surefire-plugin.
- **<build>**
- **<plugins>**
- **<plugin>**
- **<groupId>**org.apache.maven.plugins**</groupId>**
- **<artifactId>**maven-surefire-plugin**</artifactId>**
- **<version>**3.0.0-M7**</version>**
- **</plugin>**
- **</plugins>**
- **</build>**

- The surefire plugin, by default, supports all tests written in JUnit 4 and JUnit 5. It automatically detects the JUnit version and uses the appropriate runner for executing the tests.
- For JUnit 4, include the latest version of JUnit 4; for JUnit 5, include the latest version of *junit-jupiter-api* which transitively pulls in the other required dependencies.

```
<!-- For JUnit 5 -->
<dependency>
  <groupId>org.junit.jupiter</groupId>
  <artifactId>junit-jupiter-api</artifactId>
  <version>5.9.0</version>
  <scope>test</scope>
</dependency>

<!-- For JUnit 4 -->
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
```

- We are using the following test classes and methods for demo purposes.

```
public class ModuleOneTests {
        @Test public void testMethodOne() {...}
        @Test public void testMethodTwo() {...}
}
public class ModuleTwoTests {
        @Test public void testMethodOne() {...}
        @Test public void testMethodTwo() {...}
        @Test public void testMethodThree() {...}
        @Test public void testMethodFour() {...}
}
```

- **Executing JUnit Tests**
- Let us check out various ways to execute tests.
- **2.1. Execute All Test Classes and Methods**
- To execute all tests run the 'mvn test' command.
- $ mvn test
- INFO: Executing ModuleOneTests#testMethodOne
- INFO: Executing ModuleOneTests#testMethodTwo
- INFO: Executing ModuleTwoTests#testMethodOne
- INFO: Executing ModuleTwoTests#testMethodTwo
- INFO: Executing ModuleTwoTests#testMethodThree
- INFO: Executing ModuleTwoTests#testMethodFour
- [INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
- **Execute All Test Methods in Test Class**
- Pass the '-Dtest=TestClass' for executing all tests in a test class.
- $ mvn -Dtest=ModuleOneTests test
- INFO: Executing ModuleOneTests#testMethodOne
- INFO: Executing ModuleOneTests#testMethodTwo
- [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
- Pass the name of test classes in double-quotes in syntax -Dtest="TestClassOne,TestClassTwo" for executing all tests in multiple test classes.
- $ mvn -Dtest="ModuleOneTests,ModuleTwoTests" test

- INFO: Executing ModuleOneTests#testMethodOne
- INFO: Executing ModuleOneTests#testMethodTwo
- INFO: Executing ModuleTwoTests#testMethodOne
- INFO: Executing ModuleTwoTests#testMethodTwo
- INFO: Executing ModuleTwoTests#testMethodThree
- INFO: Executing ModuleTwoTests#testMethodFour
- [INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
- **Execute a Single Test Method**
- Pass the '-Dtest=TestClass#testMethod' for executing a specific test method in a test class.
- $ mvn -Dtest=ModuleOneTests#testMethodOne test
- INFO: Executing ModuleOneTests#testMethodOne
- [INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

2.4. Execute Tests with Name Matching Regex Pattern

- To execute all test methods in the application that matches a given pattern, use the regular expression wild-card to for creating a name matching pattern. For example, the following example executes all the tests with name *testMethodOne()* in all the test classes in the application.
- $ mvn -Dtest="*#testMethodOne" test
- INFO: Executing ModuleOneTests#testMethodOne
- INFO: Executing ModuleTwoTests#testMethodOne

- [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
- Note that we can create any pattern for matching test names and classes. The following command executes all the tests with names starting with test in test class *ModuleOneTests*.
- $ mvn -Dtest="ModuleOneTests#test*" test
- INFO: Executing ModuleOneTests#testMethodOne
- INFO: Executing ModuleOneTests#testMethodTwo
- [INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
- **3. Rerun Failing Tests**
- Another possible usecase can be to rerun any failing tests during the test execution. We can pass '-Dsurefire.rerunFailingTestsCount=n' parameter to rerun a failing test n times.
- The following command will run all the tests in *ModuleTwoTests* class. And each failing test will be rerun two times. We have intentionally failed the *testMethodFour()* for this example.
- mvn '-Dsurefire.rerunFailingTestsCount=2' -Dtest=ModuleTwoTests test
-
- [ERROR] Tests run: 4, Failures: 1, Errors: 0, Skipped: 0, Time elapsed: 0 s <<< FAILURE! - in com.howtodoinjava.ModuleTwoTests
- ...
- [ERROR] Failures:
- [ERROR] com.howtodoinjava.ModuleTwoTests.testMethodFour
- [ERROR] Run 1: ModuleTwoTests.testMethodFour:35
- [ERROR] Run 2: ModuleTwoTests.testMethodFour:35
- [ERROR] Run 3: ModuleTwoTests.testMethodFour:35
- ...
- [ERROR] Tests run: 4, Failures: 1, Errors: 0, Skipped: 0

Conclusion

- we have learned to execute the JUnit tests using mvn test command in various ways. We learned to execute all tests, selected tests, and even a single test. We also learned to rerun the failing tests from the command line.
-
- Develop a Maven project for sum of two numbers and test the possibilities with unit testing tool

R.M.K GROUP OF INSTITUTIONS

Maven Integration Testing

- **Overview**
- Maven is the most popular build tool in the Java space, while integration testing is an essential part of the development process. Therefore, **it's a natural choice to configure and execute integration tests with Maven.**
- In this tutorial, we'll go over a number of different ways to use Maven for integration testing and to separate integration tests from unit tests.
- **2. Preparation**
- To make the demonstration code close to a real-world project, we'll set up a JAX-RS application. This application is deployed to a server before the execution of integration tests and dismantled afterward.
- **1. Maven Configuration**
- We'll build our REST application around Jersey – the reference implementation of JAX-RS. This implementation requires a couple of dependencies:
- <**dependency**>
  <**groupId**>org.glassfish.jersey.containers</**groupId**>
  <**artifactId**>jersey-container-servlet-core</**artifactId**>
  <**version**>2.27</**version**></**dependency**><**dependency**>
  <**groupId**>org.glassfish.jersey.inject</**groupId**>
  <**artifactId**>jersey-hk2</**artifactId**>
  <**version**>2.27</**version**></**dependency**>Copy
- We'll use the Jetty Maven plugin to set up a testing environment. **This plugin starts a Jetty server during the *pre-integration-test* phase of the Maven build lifecycle, then stops it in the *post-integration-test* phase.**

- Here’s how we configure the Jetty Maven plugin in *pom.xml*:
- <**plugin**>
- <**groupId**>org.eclipse.jetty</**groupId**>
- <**artifactId**>jetty-maven-plugin</**artifactId**>
  <**version**>9.4.11.v20180605</**version**>
- <**configuration**>
- <**httpConnector**>
- <**port**>8999</**port**>
- </**httpConnector**>
- <**stopKey**>quit</**stopKey**>
- <**stopPort**>9000</**stopPort**>
- </**configuration**>
- <**executions**> <**execution**>
- <**id**>start-jetty</**id**>
- <**phase**>pre-integration-test</**phase**>
- <**goals**>
- <**goal**>start</**goal**>
- </**goals**>
- </**execution**> <**execution**>
- <**id**>stop-jetty</**id**>
- <**phase**>post-integration-test</**phase**>
- <**goals**>
- <**goal**>stop</**goal**>
- </**goals**> </**execution**>
- </**executions**></**plugin**>Copy

- When the server starts up, it'll be listening on port *8999*. The *stopKey* and *stopPort* configuration elements are used solely by the plugin's *stop* goal and their value isn't important from our perspective.
- we must set the *packaging* element in the *pom.xml* file to *war*, otherwise the plugin cannot start the server:
- <**packaging**>war</**packaging**>Copy
- **public class RestEndpoint** { @GET **public** String **hello**() { **return** "Welcome to Baeldung!"; }}Copy This is how we register the endpoint class with Jersey:
- **package** com.baeldung.maven.it; **import** org.glassfish.jersey.server.ResourceConfig;
- **public class EndpointConfig extends ResourceConfig** {
- **public EndpointConfig**()
- {
- register(RestEndpoint.class);
- }
- }
- **This descriptor must be placed in the directory */src/main/webapp/WEB-INF*** to be recognized by the server.

- **Client-Side Test Code**
- All test classes in the following sections contain a single method:
- @Test**public void whenSendingGet_thenMessageIsReturned**() **throws** IOException { **String** url = "http://localhost:8999"; **URLConnection** connection = **new URL**(url).openConnection(); **try** (**InputStream** response = connection.getInputStream(); **Scanner** scanner = **new Scanner**(response))
- {
- **String** responseBody = scanner.nextLine(); assertEquals("Welcome to Baeldung!", responseBody);
- }
- }
- As we can see, this method does nothing but sending a GET request to the web application we set up before and verifying the response.

Integration Testing in Action

- An important thing to notice about integration testing is that **test methods often take quite a long time to run.**
- As a result, we should exclude integration tests from the default build lifecycle, keeping them from slowing down the whole process each time we build a project.
- **A convenient way to separate integration tests is to use build profiles.** This kind of configuration enables us to execute integration tests only when necessary – by specifying a suitable profile.
- In the sections that follow, we'll configure all integration tests with build profiles.

- **Testing With the Failsafe Plugin**
- The simplest way to run integration tests is to use the Maven *failsafe* plugin.
- By default, the Maven *surefire* plugin executes unit tests during the *test* phase, while **the *failsafe* plugin runs integration tests in the *integration-test* phase**.
- We can name test classes with different patterns for those plugins to pick up the enclosed tests separately.
- **The default naming conventions enforced by *surefire* and *failsafe* are different, thus we just need to follow these conventions to segregate unit and integration tests.**
- The execution of the *surefire* plugin includes all classes whose name starts with *Test*, or ends with *Test*, *Tests* or *TestCase*. In contrast, the *failsafe* plugin executes test methods in classes whose name starts with *IT*, or ends with *IT* or *ITCase*.
- This is where we can find the documentation regarding test inclusion for *surefire*, and here is the one for *failsafe*.
- Let's add the *failsafe* plugin to the POM with default configuration:
- **<profile>**
- **<id>**failsafe**</id>**
- **<build>**
- **<plugins>**
- **<plugin>**
- **<artifactId>**maven-failsafe-plugin**</artifactId>** **<version>**3.1.2**</version>**
- **<executions>**
- **<execution>**
- **<goals>**
- **<goal>**integration-test**</goal>** **<goal>**verify**</goal>**
- **</goals>**
- **</execution>** **</executions>**
- **</plugin>** **</plugins>**
- **</build></profile>**

- Here we use the latest version of the *failsafe* plugin.
- With the above configuration, the following test method will be executed in the *integration-test* phase:
- **public class RestIT** { // test method shown in subsection 2.3}Copy Since the Jetty server starts up in the *pre-integration-test* phase and shuts down in *post-integration-test*, the test we have just seen passes with this command:
- mvn verify –P failsafeCopy We can also customize the naming patterns to include classes with different names:
- **<plugin>** **<artifactId>**maven-failsafe-plugin**</artifactId>** **<version>**3.1.2**</version>**
- **<configuration>**
- **<includes>**
- **<include>****/*RestIT**</include>** **<include>****/RestITCase**</include>**
- **</includes>** **</configuration>** ...**</plugin>**
- **Testing With the Surefire Plugin**
- Apart from the *failsafe* plugin, **we can also use the *surefire* plugin to execute unit and integration tests in different phases.**
- Let's assume we want to name all integration tests with the suffix *IntegrationTest*. Since the *surefire* plugin runs tests with such a name in the *test* phase by default, we need to exclude them from the default execution:
- **<plugin>** **<artifactId>**maven-surefire-plugin**</artifactId>** **<version>**2.22.2**</version>**
- **<configuration>** **<excludes>** **<exclude>****/*IntegrationTest**</exclude>**
- **</excludes>**
- **</configuration></plugin>**

- The latest version of this plugin is used.
- We've taken all test classes having a name ending with *IntegrationTest* out of the build lifecycle. It's time to put them back with a profile:
- **<profile>** **<id>**surefire**</id>** **<build>** **<plugins>** **<plugin>** **<artifactId>**maven-surefire-plugin**</artifactId>** **<version>**2.22.2**</version>**
- **<executions>** **<execution>** **<phase>**integration-test**</phase>**
- **<goals>** **<goal>**test**</goal>** **</goals>**
- **<configuration>**
- **<excludes>**
- **<exclude>**none**</exclude>**
- **</excludes>**
- **<includes>** **<include>****/*IntegrationTest**</include>** **</includes>** **</configuration>**
- **</execution>** **</executions>**
- **</plugin>** **</plugins>** **</build></profile>**

- **Instead of binding the *test* goal of the *surefire* plugin to the *test* build phase, as usual, we bound it to the *integration-test* phase.** The plugin will then kick in during the integration testing process.
- **Notice that we must set an *exclude* element to *none* to override the exclusion specified in the base configuration.**

Adding a Separate Directory for Integration Tests

- It's desirable at times to have a separate directory for integration tests. **Organizing tests this way allows us to entirely isolate integration tests from unit tests.**
- We can use the Maven *build helper* plugin for this purpose:
- **<plugin>** **<groupId>**org.codehaus.mojo**</groupId>** **<artifactId>**build-helper-maven-plugin**</artifactId>** **<version>**3.0.0**</version>** **<executions>** **<execution>** **<id>**add-integration-test-source**</id>** **<phase>**generate-test-sources**</phase>** **<goals>** **<goal>**add-test-source**</goal>** **</goals>** **<configuration>** **<sources>** **<source>**src/integration-test/java**</source>** **</sources>** **</configuration>** **</execution>** **</executions></plugin>**
- we can find the latest version of this plugin.
- The configuration we've just seen adds a test source directory to the build. Let's add a class definition to that new directory:
- **public class RestITCase** {
- // test method shown in subsection 2.3
- }
- It's time to run integration tests in this class:
- mvn verify -PfailsafeCopy The Maven *failsafe* plugin will execute methods in this test class due to the configuration we set in subsection 3.1.
- A test source directory often goes with a resource directory. We can add such a directory in another *execution* element to the plugin configuration:

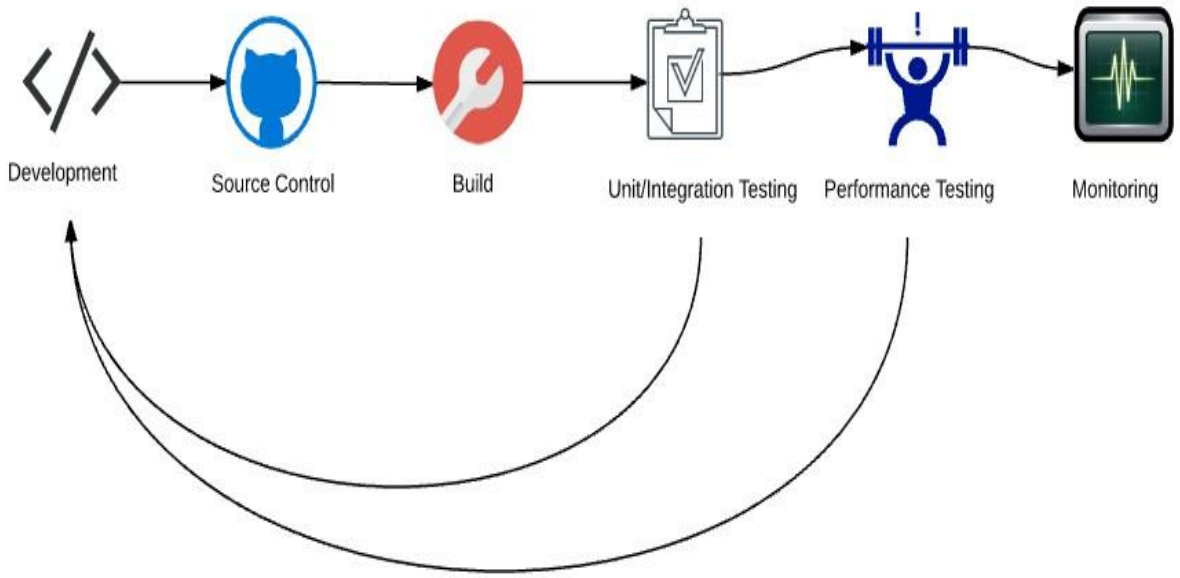
- <executions> ... <execution>
- <id>add-integration-test-resource</id>
- <phase>generate-test-resources</phase>
- <goals> <goal>add-test-resource</goal> </goals> <configuration> <resources> <resource> <directory>src/integration-test/resources</directory> </resource> </resources>
- </configuration>
- </execution></executions>

R.M.K GROUP OF INSTITUTIONS

Maven with Performance Testing

Goal:

- To create a **Continuous Delivery Pipeline** which includes **Performance Testing** process to detect any performance related issues as early as possible.



- Usually the full scale Performance Test will be done in the Staging/Pre-Production environment which could be identical to your Production environment. Code push to Staging happens after thorough QA functional/regression verification is done. So even if QA certifies the build, there is a chance that code might not work as expected at certain load which could be found only in Staging environment. This performance issue requires code change -> build -> QA verification once again which could postpone your Production push!

- So, Can we fit the performance testing into the existing continuous development & testing process to detect the performance issues as early as possible?
- The idea here is to run the performance test in the lower environments [Dev/QA] (we might have to adjust the load accordingly) & to compare the results against some baseline metrics. Note that this is NOT going to replace the full scale performance test which we will do in the Staging environment.
- Lets see how we can implement the above process by using JMeter + Maven + Jenkins.
- **Create Performance Test Script:**
- Create some basic performance test script using JMeter.
- ***Test Fragment In JMeter*** to show how to create reusable and modular test scripts.
- **Need For Test Fragment:**
- We will do performance test script in JMeter, I needed to find some approach for creating a reusable / modular test scripts. So that we could reuse the scripts for different workflows.

Example

- Lets consider a sample application which has below functionalities.
- Login
- Logout
- User Registration
- Search for Product
- Select Product
- Order Product with different payment methods
- Edit Order
- Cancel Order
- Search for User Profile

- Using above functionalities, we could come up with different workflows as shown here.
- The user access the application, searches for product, choose a product, registers, orders a product
  - ***User Registration -> Search for Product -> Select Product -> Order Product -> Logout***
- Existing user logs in, searches for product, choose a product, registers, orders a product
  - ***Login -> Search for Product -> Select Product -> Order Product -> Logout***
- Admin user logs in, searches for an user and cancels the order
  - ***Login -> Search for User -> Cancel Order -> Logout***
- We will achieved performance testing using below JMeter elements.
- Test Fragment
- Module Controller
- Parameterized Controller
- Include Controller
- **Test Fragment In JMeter:**

- ***Test Fragment*** element is a special controller which can be added directly under JMeter test plan like Thread Group. But It does nothing except holding other elements inside!! It gets executed only when it is referenced by a Module/Include controller from other Thread Groups. It acts like a library of reusable of scripts.

- **Module Controller:**
- *Module Controller* in JMeter is useful in referencing any of the logic controller in the JMeter *Test Plan*.
- For Example, We have 5 thread groups as given below in my test.
- New User Registration
- User Login & Order Creation
- User Login & Product View
- Existing Order Edit/Cancel
- User Search
- Some functionalities could be common for these thread groups. For example, User has to login to crate orders / to view the existing products.

- In the above example, you can see, Both thread groups have to have the login functionality. Whenever the login functionality changes, we need to ensure that we will update the script in both thread groups.

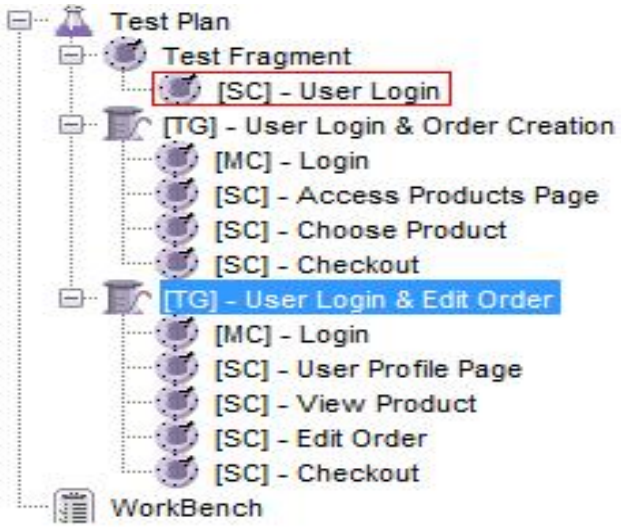
- So, instead of having duplicate simple controllers for login in both thread groups, we can add a *Test Fragment* & move the 'User Login' *Simple Controller* under the *Test Fragment*. So that it can be referenced by a *Module Controller*. [It does not have to be a *Simple Controller* always. It can be any controller].

- Now, If the script for login changes, we have to just update the 'User Login' under the *Test Fragment*. Both thread groups will work just fine.

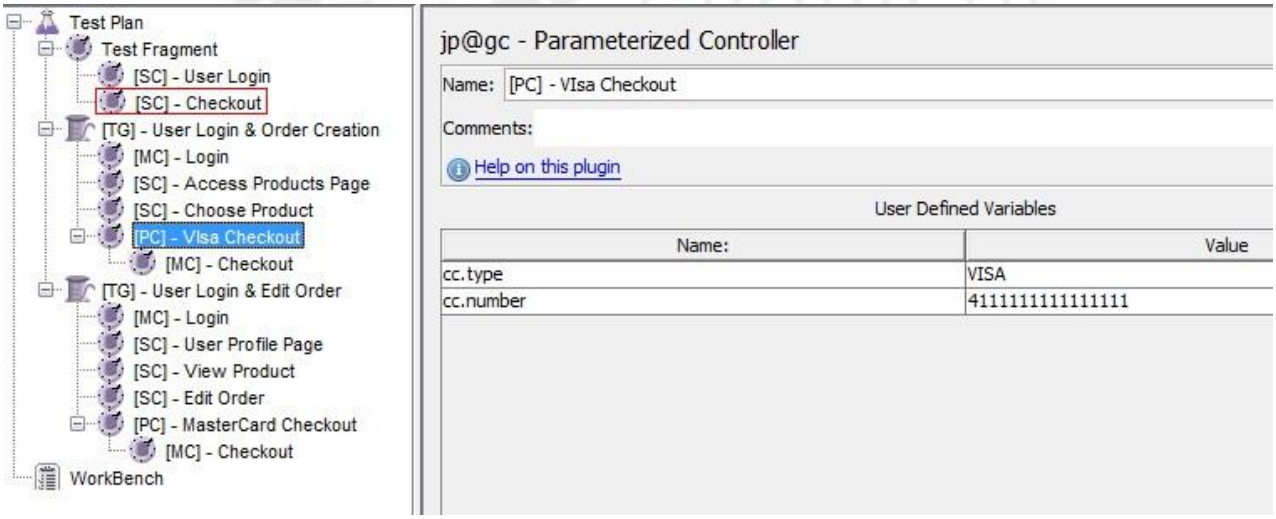
- **Parameterized Controller:**

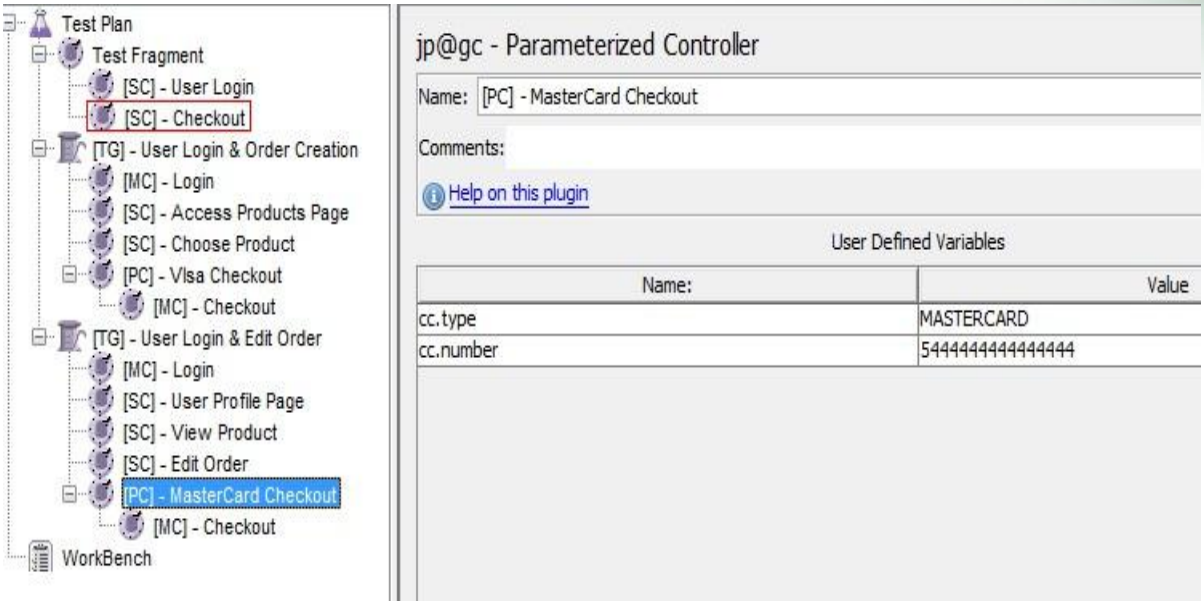
- One 'User Login' is being accessed by more than 1 *Thread Group* in the above example. Sometimes, these Thread Groups might want to use these *Simple / Transaction controller*s under the *Test Fragmen*t like a function so that they can pass different data & they expect the controllers to perform the actions based on the data passed to it.

- For example, our requirement is to use VISA credit card while ordering a new product & to use MasterCard while editing/upgrading the existing order. We can use a *Parameterized Controller* for this purpose.

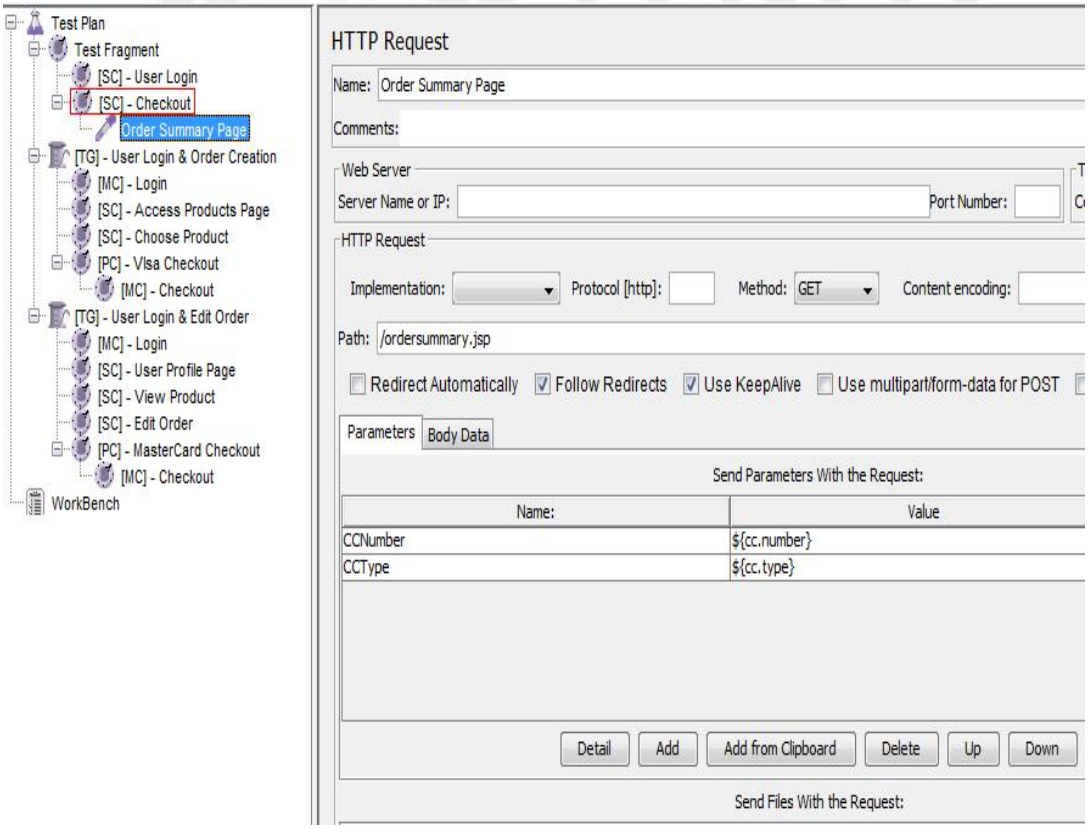


- We will add the *Parameterized Controller* first. Then we add the *Module Controller* under the *Parameterized Controller*. Now the *Module Controller* calls the 'Checkout' & passes the test data to be used in the 'Checkout'

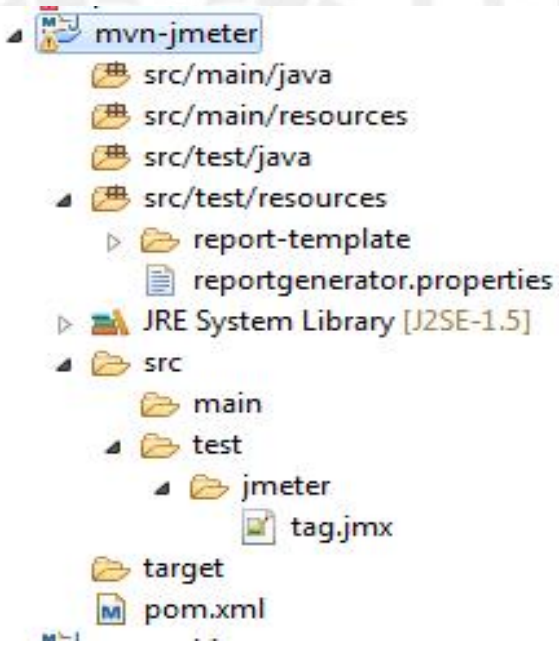




- Checkout controller will use test data passed to it while performing the requests.



- Ensure that the script can be run on any given environment like QA/PPE/Staging etc.
- Lets consider a simple JMeter test which will login, search for a product and logout.
  - we have used variables in the Thread Count, Ramp Up and Duration fields.
  - At run time, we will pass values for those variables from Jenkins.
  - To run the test in design mode/standalone mode, We use User Defined Variables as shown here. So, it takes the default values (like 1 user with 1 second ramp-up for 5 seconds test duration).
- Ensure that the scripts runs fine.
- **Running JMeter test using Maven:**
- Maven is more of a Project management tool. It is like ant – but follows a standard project structure to make our development life easier. We might not be able to create our own tasks as we did in Ant. But Maven can take care of setting up JMeter with all the dependencies to run our test.
- we create a simple mvn project in eclipse as shown here.



- Under **src/test** create a folder – **jmeter**
- Copy your jmeter jmx file & move to **src/test/jmeter**
- As part of JMeter v3.0, to create the new dashboard, ApacheJMeter.jar expects the following folders to be present in the bin location. (You can get them from your JMeter bin folder)
  - report-template
  - reportgenerator.reporties
- Keep them under **src/test/resources**
- Include below content in your pom.xml

```
<build>
    <plugins>
        <plugin>
            <groupId>com.lazerycode.jmeter</groupId>
            <artifactId>jmeter-maven-plugin</artifactId>
            <version>2.0.3</version>
            <configuration>
                <testResultsTimestamp>false</testResultsTimestamp>
                <propertiesUser>
                <threadgroup.count>2</threadgroup.count>
                <threadgroup.rampup>2</threadgroup.rampup>
                <threadgroup.duration>5</threadgroup.duration>
<jmeter.save.saveservice.thread_counts>true</jmeter.save.saveservice.thread_counts>
<jmeter.save.saveservice.thread_name>true</jmeter.save.saveservice.thread_name>
                <jmeter.save.saveservice.time>true</jmeter.save.saveservice.time>
                </propertiesUser>
            </configuration>
            <executions>
                <execution>
                <id>jmeter-tests</id>
```

```
            <phase>verify</phase>
            <goals>
              <goal>jmeter</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
      <plugin>
```

- We use jmeter-maven-plugin for executing the JMeter tests. So you do not have to download and setup JMeter in the remote machines. As long as maven is installed, it is fine.
  - Check the configuration section of this plugin for the run time properties we pass to the JMeter test.
  - More info on this is here.
- From JMeter v3.0, JMeter creates a very nice HTML dashboard for the non-gui tests. As maven has a standard folder structure we can not create folders on our own at run time as we did using ant. However maven has an ant plugin which could be very useful for this purpose.
- We can run our JMeter test using maven as shown below
- mvn clean verify
- We will see below output!

```
[INFO] --- jmeter-maven-plugin:2.0.3:configure (configure) @ mvn-jmeter ---
[INFO] ------------------------------------------------------------
[INFO]  Configuring JMeter...
[INFO] ------------------------------------------------------------
[INFO]
[INFO] <<< jmeter-maven-plugin:2.0.3:jmeter (jmeter-tests) < :configure @ mvn-jmeter <<<
[INFO]
[INFO] --- jmeter-maven-plugin:2.0.3:jmeter (jmeter-tests) @ mvn-jmeter ---
[INFO]
[INFO] ------------------------------------------------------------
[INFO]  P E R F O R M A N C E   T E S T S
[INFO] ------------------------------------------------------------
[INFO] Invalid value detected for <postTestPauseInSeconds>.  Setting pause to 0...
[INFO]
[INFO]
[INFO] Executing test: tag.jmx
[INFO] Writing log file to: C:\workspace\eclipseprojects\mvn-jmeter\target\jmeter\logs\tag.jmx.log
[INFO] Creating summariser <summary>
[INFO] Created the tree successfully using C:\workspace\eclipseprojects\mvn-jmeter\target\jmeter\testFiles\tag.jmx
[INFO] Starting the test @ Sat Nov 19 11:54:02 CST 2016 (1479578042513)
[INFO] Waiting for possible Shutdown/StopTestNow/Heapdump message on port 4445
[INFO] summary +      1 in 00:00:01 =    1.4/s Avg:     1 Min:     1 Max:     1 Err:     0 (0.00%) Active: 1 Started: 1 Finished: 0
[INFO] summary +    104 in 00:00:27 =    3.9/s Avg:     0 Min:     0 Max:     1 Err:     0 (0.00%) Active: 2 Started: 2 Finished: 0
[INFO] summary =    105 in 00:00:28 =    3.8/s Avg:     0 Min:     0 Max:     1 Err:     0 (0.00%)
[INFO] summary +    118 in 00:00:30 =    3.9/s Avg:     0 Min:     0 Max:     1 Err:     0 (0.00%) Active: 2 Started: 2 Finished: 0
[INFO] summary =    223 in 00:00:58 =    3.9/s Avg:     0 Min:     0 Max:     1 Err:     0 (0.00%)
[INFO] summary +     10 in 00:00:03 =    3.8/s Avg:     0 Min:     0 Max:     0 Err:     0 (0.00%) Active: 0 Started: 2 Finished: 2
[INFO] summary =    233 in 00:01:00 =    3.9/s Avg:     0 Min:     0 Max:     1 Err:     0 (0.00%)
[INFO] Tidying up ...    @ Sat Nov 19 11:55:02 CST 2016 (1479578102949)
[INFO] ... end of run
[INFO] Completed Test: tag.jmx
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time: 01:06 min
```

Creating HTML Report:

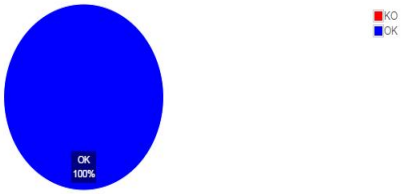
- Using **maven-antrun-plugin**, we can create a custom folder & create the new HTML dashboard. Execute below command.
- mvn pre-site
- The HTML report & charts would be created under **target\jmeter\results\dashboard.**

```
[INFO] Scanning for projects...
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] Building mvn-jmeter 0.0.1-SNAPSHOT
[INFO] ------------------------------------------------------------------------
[INFO]
[INFO] --- maven-antrun-plugin:1.3:run (default) @ mvn-jmeter ---
[INFO] Executing tasks
    [mkdir] Created dir: C:\workspace\eclipseprojects\mvn-jmeter\target\jmeter\results\dashboard
     [copy] Copying 1 file to C:\workspace\eclipseprojects\mvn-jmeter\target\jmeter\bin
     [copy] Copying 696 files to C:\workspace\eclipseprojects\mvn-jmeter\target\jmeter\bin\report-template
     [java] Writing log file to: C:\workspace\eclipseprojects\mvn-jmeter\jmeter.log
[INFO] Executed tasks
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
1.000	**500 ms**	**1 sec 500 ms**	**Total**
1.000	500 ms	1 sec 500 ms	Search
1.000	500 ms	1 sec 500 ms	Logout
1.000	500 ms	1 sec 500 ms	Login

Requests Summary



Statistics

Label	#Samples	KO	Error %	90th pct	95th pct	99th pct	Throughput	KB/sec	Min	Max
Total	**238**	**0**	**0.00%**	**0.00**	**1.00**	**1.00**	**3.96**	**1.04**	**0**	**1**
Login	80	0	0.00%	0.00	0.95	1.00	1.34	0.35	0	1
Logout	78	0	0.00%	0.00	1.00	1.00	1.34	0.35	0	1
Search	80	0	0.00%	0.00	0.00	1.00	1.34	0.35	0	1

Running Distributed Testing using Maven:

Under the **configuration** section of **jmeter-maven-plugin**, add below section for JMeter distributed testing.

```
<remoteConfig>
  <startServersBeforeTests>true</startServersBeforeTests>
    <serverList>server1,server2</serverList>
    <stopServersAfterTests>true</stopServersAfterTests>
</remoteConfig>
```

R.M.K GROUP OF INSTITUTIONS

Report Generation and Configuration

- **Configuring Reports**
- Since version 1.1, all Maven reports will be included by default in the generated PDF. You should configure the <reporting/> section of your POM similar than the site plugin
- <project> ... <reporting> <plugins> <plugin> <groupId>org.apache.maven.plugins</groupId> <artifactId>maven-project-info-reports-plugin</artifactId> <version>2.1.2</version> <reportSets> <reportSet> <reports> <report>project-team</report> ... </reports> </reportSet> </reportSets> </plugin> ... </plugins> </reporting> ... <project>

- **Notes:**
- to exclude the reporting generation inside the PDF, you should add -DincludeReports=false in the command line.
- only internal reporting plugins will be added in the PDF, external reporting plugins like Javadoc will be skipped.

- **Enhancements**
- Having many reports increases the build time, so it is recommended to select only the wanted reports to be included in the PDF. It is recommended to define a reporting profile in your pom, similar to the following:

- <project> ... <profiles> <profile>
- <id>pdf</id> <reporting> <plugins> <plugin> <artifactId>maven-project-info-reports-plugin</artifactId> <version>2.1.2</version>
- <reportSets> <reportSet>
- <reports> <report>cim</report>
- <!-- take too long time <report>dependencies</report> --> <report>dependency-convergence</report> <report>dependency-management</report>
- <!-- already present <report>index</report> --> <report>issue-tracking</report> <report>license</report> <report>mailing-list</report> <report>plugin-management</report> <report>plugins</report> <report>project-team</report> <report>scm</report>
- <report>summary</report>
- </reports> </reportSet> </reportSets> </plugin> </plugins> </reporting>
- <build> <defaultGoal>pdf:pdf</defaultGoal> <plugins> <plugin>
- <groupId>org.apache.maven.plugins</groupId> <artifactId>maven-pdf-plugin</artifactId> <version>1.6.1</version>
- </plugin> </plugins> </build>
- </profile> ... </profiles> ... <project>

Generated Reports

- This document provides an overview of the various reports that are automatically generated by Maven . Each report is briefly described below.

Document	Description
Surefire Report	Report on the test results of the project.
Checkstyle	Report on coding style conventions.
PMD	Verification of coding rules.
Source Xref	HTML based, cross-reference version of Java source code.
Test Source Xref	HTML based, cross-reference version of Java test source code.
Tag List	Report on various tags found in the code.
Javadoc	Javadoc API documentation.
Test Javadoc	Test Javadoc API documentation.
FindBugs	Generates a source code report with the FindBugs Library.
Plugin Documentation	This report provides goals and parameters documentation of a plugin.
Invoker Build Results	The results of the Maven invocations.
L10n Status	This report provides information about the completeness of localization.

Jenkins Configuration

1.1 Initial Setup

After installing **Jenkins**, the following configuration steps are mandatory:

a) Unlock Jenkins

Access Jenkins at http://<server>:8080

Enter the **initial admin password** from:

/var/lib/jenkins/secrets/initialAdminPassword

b) Install Plugins

Suggested Plugins: Git, Pipeline, Blue Ocean, Maven, Docker

Custom Plugins: Install based on project needs (Kubernetes, Slack, SonarQube)

1.2 Global Configuration

Accessible via:

Manage Jenkins → Configure System

a) Jenkins URL

Required for webhooks and email notifications

Example:

http://jenkins.company.com:8080

b) Executors

Defines how many jobs can run in parallel

Example:

Small server → 2 executors

Large server → 8–16 executors

1.3 Tool Configuration

Manage Jenkins → Global Tool Configuration

Tool	Purpose
JDK	Java builds
Maven	Java project builds
Gradle	Android / Java builds
NodeJS	Frontend builds
Docker	Containerization

Jenkins Configuration

1.4 Security Configuration

Manage Jenkins → Security

Authentication:

- Jenkins local DB
- LDAP / Active Directory

Authorization:

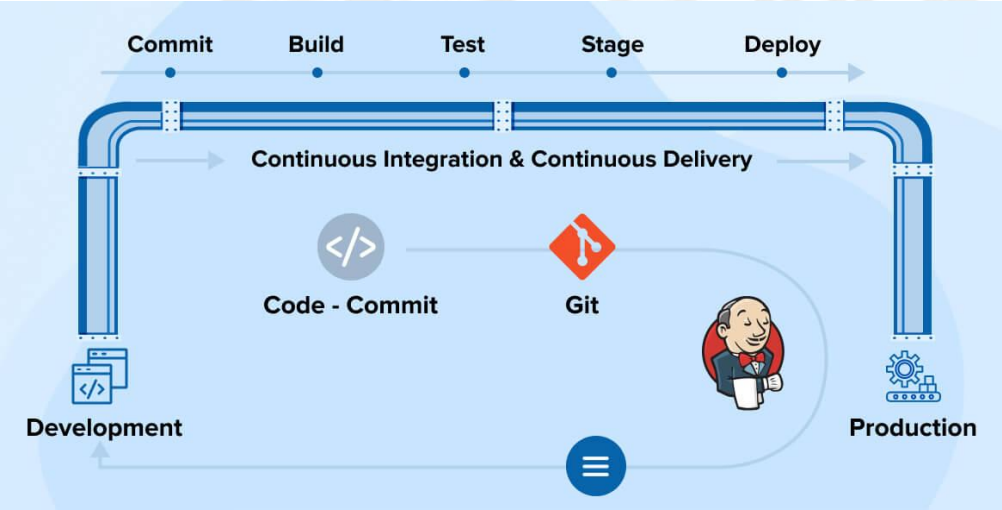
- Role-Based Strategy

Best Practices:

- Disable anonymous access
- Use credentials store
- Enable HTTPS

:

II. Continuous Delivery (CD) in Jenkins



Continuous Delivery ensures that:

Code is **always in a deployable state**

Deployment can be done **on demand or automatically**

Jenkins Role in CD

Jenkins automates:

Build

Test

Package

Deploy

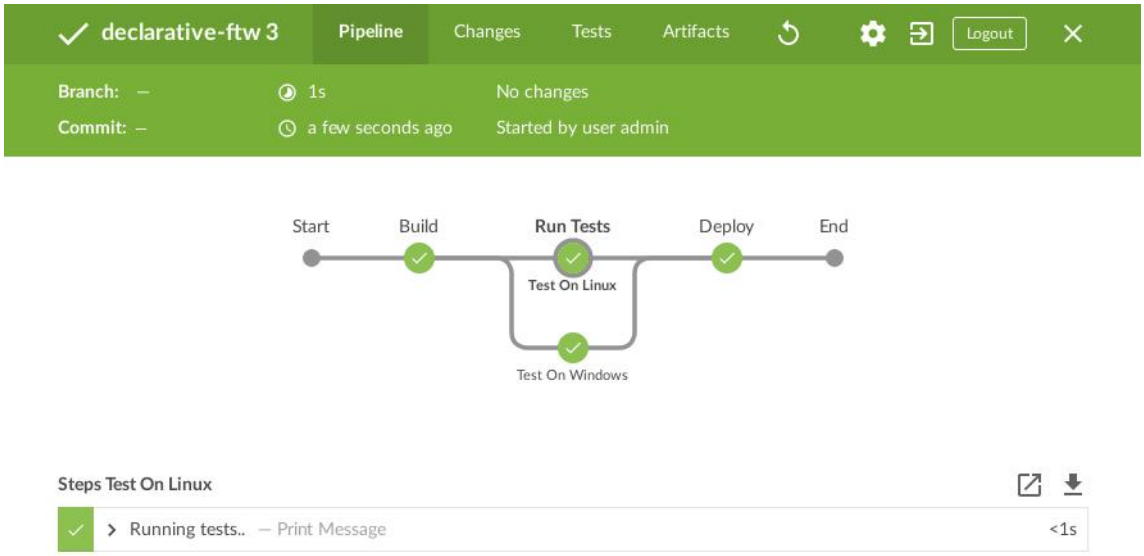
Jenkins Configuration

CD Deployment Targets
Dev
QA
UAT
Production

Example CD Flow:

Git Commit → Jenkins Build → Tests → Docker Image → QA Deployment → Approval → Production Deployment

Jenkins Pipelines



What is a Jenkins Pipeline?
A **Pipeline** is a set of **automated steps defined as code** to perform CI/CD.
Defined using a Jenkinsfile.

Part A

1. **What is the purpose of the `pom.xml` file in Maven**

The `pom.xml` file is the core configuration file in Maven that defines project dependencies, build settings, plugins, and **more**.

2.What is Maven?

Maven is a build automation tool primarily used for Java projects. It helps in dependency management, project builds, and reporting.

3.What are Maven goals?

Goals are **specific** tasks executed by Maven, such as `compile`, `test`, and `package`.

4.How do you define a dependency in Maven?

By specifying it in the `<dependencies>` section of `pom.xml` with `<groupId>`, `<artifactId>`, and `<version>` tags.

```
<dependency>
   <groupId>junit</groupId>
   <artifactId>junit</artifactId>
   <version>4.13.2</version>
   <scope>test</scope>
</dependency>
```

5. What is the Maven repository?

A Maven repository is a storage location for project dependencies, plugins, and artifacts. Repositories can be local, central, or remote.

Part A

7. Which plugin is used for unit testing in Maven?

The Surefire Plugin is used for running unit tests.

8. How do you configure the Surefire Plugin in Maven?

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.0.1</version>
</plugin>
```

9. What is the default directory for unit test classes in Maven?

src/test/java

10.How do you execute tests in Maven?

By running the command `mvn test`.

11.How can you skip tests during the build process?

Use the command mvn install -DskipTests=true

12.What plugin is commonly used for integration testing in Maven?

The Failsafe Plugin is used for integration testing.

13.How do you configure the Failsafe Plugin in Maven?

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-failsafe-plugin</artifactId>
    <version>3.0.1</version>
</plugin>
```

14. What is the difference between Surefire and Failsafe plugins?

Surefire runs unit tests, while Failsafe runs integration tests.

Part A

17. How do you specify integration test classes?

Integration test classes should follow the naming convention *IT.java.

18. How do you run integration tests with Maven?

Use the command mvn verify

19. Which tool can you integrate with Maven for performance testing?

Tools like JMeter or Gatling can be integrated for performance testing.

20. How do you add the Maven JMeter plugin?

```
<plugin>
   <groupId>com.lazerycode.jmeter</groupId>
   <artifactId>jmeter-maven-plugin</artifactId>
   <version>3.7.0</version>
</plugin>
```

21. What command is used to execute JMeter tests with Maven?

mvn jmeter:jmeter

22. How do you generate performance test reports using Maven?

By configuring the reporting options in the plugin configuration.

23. Why is performance testing important in Maven builds?

To ensure the application meets performance requirements under various loads.

24. Which plugin is used to generate test reports in Maven?

The Surefire Report Plugin..

Part B

- **Explain what is Maven? How does it work?**
- **List out what are the aspects does Maven Manages?**
- **Explain what is Maven Repository? What are their types?**
- **Why Maven Plugins are used?**
  - Maven plugins are used to
  - Create a jar file
  - Create war file
  - Compile code files
  - Unit testing of code
  - Documenting projects
  - Reporting

- **List out what are the build phases in Maven?**
- **Explain installation steps for Maven project**
- **Explain in details about Maven with Unit testing with Example**
- **Explain in details about Maven with Integration testing with Example**
- Explain how reports generated using Maven software tool.
- **Explain how performance testing is performed in Maven projects with Example**

SUPPORTIVE ONLINE COURSES – UNIT 3

- https://swayam.gov.in/explorer?searchText=software+engineering
- https://www.coursera.org/learn/software-processes
- https://www.coursera.org/specializations/software-development-lifecycle
- https://www.coursera.org/learn/software-design-development-life-cycle

R.M.K GROUP OF INSTITUTIONS

REAL TIME APPLICATION- UNIT 3

- As per the KPMG Survey, on average, about 70 % of all IT-related projects fail to meet their objectives, and one of the main reason is the selection of the wrong software development process/model.
- A software house owner explained in an agile conference "We were facing problems like over commitment and under delivery, unsatisfied customers, missing deadlines and unhappy team. We switched our development model, and it all turned out to be ok."
- Just selecting a model that suits your criteria can avoid these problems.
- For example, if requirements are all clear and written down, you might have to choose a different model if requirements are not clear and constant changes are expected.

Waterfall

- Probably the oldest and most straightforward SDLC (software development life cycle) development model, Waterfall follows a sequential model (like a waterfall) with requirements analysis, design, coding, testing and implementation in such a manner that the development does not move to next phase until the previous phase is completed. That is why it is called the linear sequential software development life cycle.

Iterative

- Repetition is the keyword for the iterative model. Instead of having clear and known requirements or waiting for them, a team starts development on the known features, tests and then evaluate further requirements, develops, tests and so on until the whole thing is done.

Agile

- Agile, the most widely used software development methodology is Agile, has become the industry standard, be it software development, App development or Game development.

SOFTWARE PROJECT MANAGEMENT

Software project management refers to the branch of project management dedicated to the planning, scheduling, resource allocation, execution, tracking and delivery of software and web projects.

Project management in software engineering is distinct from traditional project management in that software projects have a unique lifecycle process that requires multiple rounds of testing, updating, and customer feedback. Most IT-related projects are managed in the agile style, in order to keep up with the increasing pace of business, and iterate based on customer and stakeholder feedback.

❁ ProWorkflow

ProWorkflow is a web-based project management software that enables users to manage tasks and projects, track time, organize contacts, and generate reports for their business. It is a productivity application that provides a comprehensive set of features, yet still easy to use for all members of the team. Aside from the great tools and functionality from the software, customers also enjoy free quality support through consultations and trainings that help get their businesses up and running.

❁ Jira

Jira is an agile project management software used by development teams to plan, track, and release software. It is a popular tool designed specifically and used by agile teams. Aside from creating stories, planning sprints, tracking issues, and shipping up-to-date software, users also generate reports that help improve teams, and create their own workflows. As part of Atlassian, it integrates with many tools that enable teams to manage their projects and products from end to end. Jira Software is built for every member of your software team to plan,track, and release great software. Every team has a unique process for shipping software. Use an out-of-the-box workflow, or create one to match the way your team works.

ASSESSMENT SCHEDULE

Tentative schedule for the Assessment During 2025-2026 Even semester

S.NO	Name of the Assessment	Start Date	End Date	Portion
1	UNIT TEST 1			UNIT 1
2	IAT 1			UNIT 1 & 2
3	UNIT TEST 2			UNIT 3
4	IAT 2			UNIT 3 & 4
5	MODEL			ALL 5 UNITS

R.M.K GROUP OF INSTITUTIONS

REFERENCES:

- Jennifer Davis and Ryn Daniels, Effective DevOps: Building a Culture of Collaboration, Affinity, and Tooling at Scale, First Edition, O'Reilly Publications.
- Gene Kim, Patrick Debois et al., The DevOPS Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations, IT Revolution Press; Illustrated edition, 2016.

R.M.K GROUP OF INSTITUTIONS

MINI PROJECT SUGGESTIONS

1.Student Mark Analysis System

Problem Statement:

Computerized Student Mark Analysis System analysis the marks obtained by the students and generate various reports of an Institution.

- It needs to maintain the record of all the students and staff of an Institution.
- It should be able to perform basic operations such as:
  - Add New Student and Staff details
  - Update Student and Staff Details
  - Delete Student and Staff Details.
  - View Results

Generate Reports:

- Branch-wise Student Report
- Grade-wise Student Report
- Subject-wise Student Report
- Year-wise Student Report

R.M.K GROUP OF INSTITUTIONS

MINI PROJECT SUGGESTIONS

- **Exercise 1:**
- Install Jenkins
- Configure Jenkins for Maven, Java, GitHub,SonarQube and SonarScanner
- Setup Continuous Integration on Jenkins for the above said project and show case the build stability in the form of
  - Build Stability
  - Test Case Success / Regression
  - Code Quality Check using SonarQube
- Points to be observed: -
- Build Stability
- Finger Prints
- Test Case Results
- Workspace
- Application Logs
- Jenkins Pre / Post Actions
- Email Notifications
- Sonar Qube Report Analysis

R.M.K GROUP OF INSTITUTIONS

- **Exercise 2:**
- Install Groovy
- Create a Pipelines in Jenkins which will perform following steps
  - Configure Java/Maven/Private Repo
  - Git Clone of the above said project
  - Maven Build
  - Maven Test
  - Sonar Scanner (if quality if OK then proceed, else exit with error message)
  - Repo Setup and Install Libraries in the Repo
  - Docker Clean Containers
  - Docker Clean Images
  - Docker Build
  - Docker Run Container
- Points to be observed: -
- Build Stability
- Finger Prints
- Test Case Results
- Workspace
- Application Logs
- Jenkins Pre / Post Actions
- Email Notifications
- Sonar Qube Report Analysis
- Time Taken for each Steps
- Process Refinement

R.M.K GROUP OF INSTITUTIONS

MINI PROJECT SUGGESTIONS

2. Quiz System

Problem Statement:

Online Quiz System allows different categories of candidates to attend online test on various domains according to their required skill set.

- It needs to maintain the record of all the participants by System Administrator.
- It should be able to perform basic operations such as:
  - Registration/SignIn
  - Update Participant Details.
  - Delete Participant Details.
  - Add domain questions for various levels (Easy/Moderate/Difficult)
  - Attend Quiz & Display Score.
  - Generate Participant Report.

R.M.K GROUP OF INSTITUTIONS

3.Ticket Reservation System

Problem Statement:

Online Ticket Reservation System allows the user to book/cancel the tickets online from their place where the internet connectivity is feasible without standing in the queue for a long time.

- It needs to maintain the record of all the train details, station details and passenger details of a particular train.
- It should be able to perform basic operations such as:
  - Registration/SignIn
  - Add/Update/Delete Train Details
  - Check Availability of seats
  - Book Tickets
  - Cancel Tickets
  - Payment

4.Expert System

Problem Statement:

Expert System is used to identify an ailment by its symptoms and prescribes an appropriate medicine corresponding to the disease. It is well known that developing countries are facing lots of shortage of medical expertise.

- It needs to maintain the record of all the symptoms and its prescribed medicines.

- It should be able to perform basic operations such as:
  - Registration/SignIn
  - Add/Update/Delete Symptom Details
  - Querying about Symptoms

R.M.K GROUP OF INSTITUTIONS

5.Course Registration System

Problem Statement:

Computerize Online Course Registration System and register students for various courses based on the availability of seats and course instructor and generate various reports.

- It needs to maintain the record of all the courses, seats available, course instructors and duration.
- It should be able to perform basic operations such as:
  - Add new Courses and Course Instructors details
  - Update Courses and Course Instructors details
  - Delete Courses and Course Instructors details
  - View Registration status
  - Generate Reports:
  - Course-wise Registration Report
  - Courses Availability Report
  - Course Instructor Availability Report
  - Year-wise Registration Report
  - Courses and Course Instructors Report

6.Platform assignment System for the Trains in a Railway Station

Problem Statement:

Computerize Platform assignment for the trains arriving to the railway station and generate various reports.

- It needs to maintain the record of Number of platforms, Number of trains arriving and leaving the station and time details.
- It should be able to perform basic operations such as:
  - Add new platform and train details
  - Update platform and train details
  - Delete platform and train details
  - View platform status
  - Generate Reports:
  - Trains arriving and leaving the station Report
  - Platform assignment details Report on a particular date and between dates
  - Platform available status Report for any instant

Thank you

Disclaimer:

This document is confidential and intended solely for the educational purpose of RMK Group of Educational Institutions. If you have received this document through email in error, please notify the system manager. This document contains proprietary information and is intended only to the respective group / learning community as intended. If you are not the addressee you should not disseminate, distribute or copy through e-mail. Please notify the sender immediately by e-mail if you have received this document by mistake and delete this document from your system. If you are not the intended recipient you are notified that disclosing, copying, distributing or taking any action in reliance on the contents of this information is strictly prohibited.